Employing Metaprogramming: A Macro-Driven Strategy to Overcome Compiler-Imposed Limitations on Specialization

Davide Cerato
Id. Number: 984481

BSc in Computer Science

Advisor: Prof. Walter Cazzola
Co-Advisor: Mr. Federico Cristiano Bruzzone





UNIVERSITÀ DEGLI STUDI DI MILANO
Computer Science Department
ADAPT-Lab

Academic Year 2024–2025

COMPUTER SCIENCE DEPARTMENT • ADAPT-LAB

Contents

1 Introduction

Premise A core challenge in high-performance systems programming is achieving zero-cost abstraction through generic programming and polymorphism while allowing type-specific performance optimizations. This concept is known as *specialization* and it's a common aspiration in advanced type systems. This challenge is particularly acute in Rust, a modern systems programming language designed for safety, concurrency and performance. Its trait system enable polymorphism and generic programming without incurring runtime overhead, as the compiler typically performs monomorphization, ensuring high performance across the board. However, for maximum efficiency, there are instances where a generic function could benefit significantly from a custom, highly optimized implementation when dealing with specific types / conditions. The ability to provide such a conditional, type-specific implementation for a trait method is known as Trait Specialization.

State of the Art Many programming languages have explored various mechanisms to achieve specialization, with varying degrees of success and complexity. Without native support for specialization, in many cases the common practice is to rely on various complex workarounds which often result in cumbersome code, poor compiler diagnostics or insufficient optimization compared to true specialization. In 2015, the Rust community introduced an experimental feature called *trait specialization* [**?**], which soon received widespread interest, but up to now is still confined to the *Nightly* channel, after its stabilization attempts stalled due to various design and implementation challenges.

Limitations A general obstacle for adopting powerful, cutting-edge language features is the tension between innovation and stability, often manifested in experimental versus production-ready compiler versions. This is true for specialization as well, forcing library authors and application developers to make an undesirable trade-off: either forgo the performance and elegance improvements offered by specialization to maintain stability or commit to volatile and constantly changing unstable versions. For example, the primary and most critical limitation facing the Rust ecosystem is indeed the Nightly-Stable divide regarding specialization (the difference between these channels is explained in Section **??**). While the feature offers significant potential for optimization and cleaner trait designs, its unstable status prevents its adoption in commercial, production-ready or long-term supported codebases.

Problem Statement While the concept of specialization is a crucial tool for optimizing generic code in any performance-oriented language, its practical application is often gated by compiler maturity and stability concerns. The lack or instability of specialization in programming languages creates a gap between what is theoretically possible for optimization and what is practically available to the vast majority of the community, putting a performance ceiling on stable code. Specifically for Rust, given its performance focus and the significant utility of trait specialization, the central problem is the lack of a robust, idiomatic and performance-preserving mechanism for implementing trait specialization in stable code.

Proposal To address the general limitations of specialization features in current stable ecosystems, this thesis proposes an innovative approach to the matter by moving the monomorphization and specialization logics to an earlier stage in the compilation process. To prove the effectiveness of this approach, we introduce a metaprogramming-based technique to simulate the behavior of compiler-implemented specialization. By utilizing the metaprogramming capabilities of the language, we can analyze the input and generate the necessary boilerplate and type-checking logic at compile time, effectively allowing a user to define specialized implementations in stable code. This approach (which can be applied to most languages with metaprogramming support) avoids runtime overhead entirely and circumvents the restrictions imposed by unstable compiler features.

Contributions This work provides the following key contributions. A Novel Stable Specialization Technique: the design and implementation of a new, purely metaprogramming-based approach for achieving specialization. Implementation of Reusable Tools: the creation of a Rust procedural macro library that provides an alternative to nightly specialization and an analysis tool that evaluates real-world public Rust codebases to identify common patterns where this stable specialization mechanism could yield substantial benefits.

Structure The remainder of this thesis is structured as follows. Chapter **??** (**??**) illustrates the tool in action through a concrete example. Chapter **??** (**??**) provides foundational knowledge on Rust, dispatch techniques and metaprogramming. Chapter **??** (**??**) details the design and architecture of the specialization macro tool. Chapter **??** (**??**) presents the experimentation methodology and results, including benchmarks and ecosystem analysis. Chapter **??** (**??**) reviews existing solutions for specialization in Rust and related languages. Chapter **??** (**??**) outlines potential extensions and further research avenues. Finally, Chapter **??** (**??**) summarizes the findings and concludes the thesis.

2
Running Example

The simplest form of specialization using the tool is the following. We'll show the example in Rust, but the approach is general and can be applied to other languages with similar features.

The default implementation does not need any change compared to standard code. The specialized implementation instead, is annotated with the attribute macro `#[when(...)]`, that lets us specify the conditions under which that implementation should be used. In this case, we want to specialize the implementation of `MyTrait` for the type `i32`, so we write `#[when(T = i32)]`. This will create a specialized version of the `trait` and of the `impl`, through a process we'll be calling *monomorphic specialization*.

After doing this, we also need to instruct the method calls to use the correct version of the method. This is done by using the `spec!` macro, which takes as first argument the method call and as second argument the type to specialize for. As better explained in Chapter **??**, this is required because in the macro expansion phase the compiler doesn't give us enough information to automatically determine the types of the variables.

The complete code is shown in Listing **??**, where you can see the macros in action. As you can probably understand, this approach preserves the soundness of the type system, since after the monomorphization process (see Listing **??**) we have two completely separate versions of the trait and its implementation, that the compiler can easily distinguish and type check independently.

```
1  trait MyTrait { fn my_method(&self); }
3  impl<T> MyTrait for T {
4      fn my_method(&self) {
5          println!("Generic implementation");
6      }
7  }
9  #[when(T = i32)]
10 impl<T> MyTrait for T {
11     fn my_method(&self) {
12         println!("Specialized for i32");
13     }
14 }
16 fn main() {
17     spec! { 10.my_method(); i32 };   // prints "Specialized for i32"
18     spec! { 10.5.my_method(); f64 }; // prints "Generic implementation"
19 }
```

Listing 1. *Example of the tool usage.*

```rust
trait MyTrait { fn my_method(&self); }

trait MyTrait_i32 { fn my_method(&self); }

impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}

impl MyTrait_i32 for i32 {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}

fn main() {
    <i32 as MyTrait_i32>::my_method(&10); // prints "Specialized for i32"
    <f64 as MyTrait>::my_method(&10.5);   // prints "Generic implementation"
}
```

Listing 2. *Example of the code after our macros' expansion.*

In addition to using the two macros, we need to instruct the compiler to allow us to define and use the macros correctly independently from the order of their definition. In Rust this is done with *build scripts*, as shown in Listing **??**.

```rust
fn main() {
    spec_trait_order::handle_order();
}
```

Listing 3. *Example of the minimal content of the* `build.rs` *file.*

With this simple example, we have shown how to use the tool to define a specialized implementation of a trait method for a specific type. In the next chapters, we will explore more complex scenarios, including multiple specializations, constraints on generic parameters and how the tool handles these cases internally.

3 Background

3.1 The Rust Programming Language.

Rust is a system programming language that focuses on safety, speed and concurrency. It ensures memory safety without garbage collection, meaning pure Rust programs are free from null pointer dereferences and data races.

Ownership and Borrowing Rust's ownership system—integrated into its type system—is inspired by *linear logic* [**?**, **?**] and *linear types* [**?**, **?**], enforcing that each piece of memory (a value) has a single *owner* (the variable binding) at any time [**?**, **?**]. When the owner goes out of scope, the memory is automatically deallocated, enabling user-defined destructors and supporting the *resource acquisition is initialization* (RAII) pattern [**?**]. Ownership can be transferred (*moved*) or temporarily shared (*borrowed*) through references. Rust supports two types of borrows: multiple *immutable* borrows or a single *mutable* borrow, but never both at the same time. These constraints, enforced by the compiler through *borrow checking*, guarantee memory safety and prevent dangling pointers by ensuring that reference lifetimes never outlive their owners. To support low-level operations, Rust provides `unsafe` blocks, where the compiler's safety guarantees are suspended and the burden of avoiding undefined behavior falls on the programmer. Outside of these blocks, Rust enforces strict safety, making undefined behavior impossible in safe code.

Structs, Traits and Impls Structs (`struct`) and enums (`enum`) are the primary composite data types in Rust, enabling the creation of complex data structures with named fields and variant types, respectively.

```
struct MyStruct {
    field1: i32,
    field2: String,
}
```

Listing 4. *Example of a Rust struct.*

```
enum MyEnum {
    Variant1,
    Variant2(i32),
}
```

Listing 5. *Example of a Rust enum.*

Rust's trait system is a central mechanism for expressing polymorphism and shared behavior across types [**?**, **?**]. Traits (`trait`) define a set of method signatures that types can implement, enabling abstraction over operations without committing to a concrete type. Implementation blocks (`impl`) associate specific types with traits, providing

concrete method definitions that satisfy the trait's contract. Rust also supports *inherent implementations*, where methods are defined directly on a type without requiring a trait, allowing type-specific functionality and constructor-like associated functions.

```
trait MyTrait { fn do_something(&self) }
impl MyTrait for i32 {
    fn do_something(&self) {}
}
```

Listing 6. *Example of a Rust trait impl.*

```
struct MyStruct;
impl MyStruct {
    fn do_something(&self) {}
}
```

Listing 7. *Example of a Rust inherent impl.*

By combining *traits*, *trait bounds* and *impl blocks*, Rust achieves ad hoc polymorphism [**?**, **?**] and generic programming [**?**, **?**] while preserving static guarantees and enabling the compiler to resolve method calls efficiently at compile time. Trait-based design is tightly integrated with Rust's ownership and borrowing model, ensuring that polymorphic abstractions do not compromise memory safety.

Software Variability Rust supports software variability through its attribute system, particularly the `cfg` attribute, which allows conditional compilation based on specified configuration predicates A feature in Rust is defined by all *terms* annotated with the same *config name* in a `cfg` *outer attribute*. The inclusion of a feature in a product depends on the evaluation of a *configuration predicate* at compile time, e.g. `#[cfg(any(unix, windows))]`. *Cargo*, Rust's package manager, lets developers declare features using *config names* in the `Cargo.toml` file, combining them with logical operators into *configuration predicates*. Feature dependencies, i.e., *cross-tree constraints* of *feature models* [**?**, **?**, **?**], can also be specified in the same file.

Release Channels Rust is distributed through three release channels: *stable*, *beta* and *nightly*. These channels differ in maturity guarantees and access to experimental language features [**?**]. The *stable* channel prioritizes reliability and backwards compatibility: only fully tested features are enabled, ensuring that libraries and applications compiled on stable continue to work across future releases. The *beta* channel serves as a staging ground for upcoming stable releases, allowing developers to test new features before they reach stable. In contrast, the *nightly* channel is updated daily and exposes experimental compiler internals, unstable language constructs and opt-in features (guarded by `#![feature(...)]` attributes). These features include advanced compiler hooks, low-level intrinsics and incomplete language extensions such as *specialization* and are available exclusively on nightly because their behavior is still under active design and soundness evaluation. Although nightly enables powerful experimentation, it provides no stability guarantees: features may change, be redesigned or removed without deprecation. For this reason, production-grade Rust codebases overwhelmingly target the stable channel, limiting developers from relying on certain expressive but unstable mechanisms.

3.2 Dispatch Techniques

Modern programming languages employ a variety of dispatch strategies at run-time or compile-time to determine which function or method is executed. These strategies fundamentally influence a language's expressiveness, performance guarantees and capacity for static reasoning [?, ?]. Rust integrates several dispatch paradigms to reconcile expressive abstraction with predictable performance, adhering to the principle of *zero-cost abstraction* [?, ?]. Most decisions are resolved statically, enabling the compiler to generate optimized machine code without runtime overhead [?, ?]. At the same time, Rust offers controlled forms of dynamic dispatch where late binding is essential, such as trait objects [?]. Beyond these core mechanisms, techniques like inlining heuristics [?], function cloning [?] and trait-based overloading [?] enable the compiler to further optimize code based on contextual information [?, ?]. Together, Rust's dispatch mechanisms provide a flexible yet robust model that preserves safety and performance.

Monomorphization Monomorphization [?] is the process by which the compiler generates concrete, type-specific versions of generic functions, methods and data structures. When a generic function is instantiated with a particular type parameter, the compiler produces a specialized version of the function in the final binary. This approach mirrors C++ template instantiation [?], enabling aggressive optimizations such as inlining, constant propagation and dead-code elimination [?, ?, ?].

```cpp
template <typename T>
T add(T a, T b) {
    return a + b;
}
int main() {
    // 1. Instantiation with int
    int x = add<int>(5, 3);
    // 2. Instantiation with double
    double y = add<double>(5.5, 3.2);
    return 0;
}
```

```cpp
int add_int(int a, int b) {
    return a + b;
}
double add_double(double a, double b) {
    return a + b;
}
int main() {
    int x = add_int(5, 3);
    double y = add_double(5.5, 3.2);
    return 0;
}
```

Listing 8. *Monomorphization process in C++: on the left, a generic function `add<T>` is defined and instantiated with two different types (`int` and `double`); on the right, the compiler generates two specialized versions of the function (`add_int` and `add_double`) and uses them accordingly.*

Because all type information is known at compile time, monomorphized functions avoid virtual dispatch overhead and allow the compiler to reason precisely about memory layout [?]. The primary trade-off is *code bloat*: large applications with many generic instantiations may experience increased binary size. Nevertheless, Rust's monomorphization strategy is central to its performance model, making generic abstractions effectively as fast as handwritten, type-specific code.

Overloading Overloading allows multiple functions or methods to share the same name but differ in parameter types or counts. Rust does not support traditional func-

tion overloading by parameter types as seen in languages like C++ or Java. Instead, it employs a unified mechanism based on *trait resolution* and *associated functions* [?]. Overloading emerges indirectly: different types may provide different implementations of the same trait methods and users may introduce trait bounds to guide method selection. This form of ad hoc polymorphism resembles *type classes* in Haskell [?, ?], where method resolution is determined by the combination of the type and the trait. Rust enforces coherence to guarantee that the compiler can unambiguously select an implementation at compile time [?, ?], preventing conflicts caused by independently defined overlapping instances. In addition, traits can define associated functions that behave similarly to static methods, allowing for API designs that mimic overloading while remaining type-safe and deterministic. This approach ensures predictable dispatch behavior, maintains clarity in method resolution and avoids the complexities of multi-parameter or ambiguous overload sets common in other languages [?].

Specialization Specialization is an advanced dispatch mechanism that allows more specific trait implementations to refine or override default behavior. In Rust, specialization aims to resolve long-standing expressiveness gaps in the trait system by enabling prioritized selection among overlapping implementations. This is particularly useful when a general-purpose implementation exists (often expressed as a *default* method in a trait) but certain types require optimized or semantically distinct behavior [?]. The feature builds on Rust's *coherence* and *orphan rules*[?], ensuring that overlapping implementations maintain type-safety and global consistency [?]. Although as of today specialization remains unstable and experimental (through the `#![feature(specialization)]` or `#![feature(min_specialization)]` flags [?, ?]), it provides a foundation for advanced library design patterns, such as optimized container types, numerical traits and fallback-based APIs [?]. The mechanism is conceptually related to multi-methods and ad hoc polymorphism in other languages [?, ?], but Rust restricts its form to preserve determinism and avoid ambiguity in resolution that have yet to be fully addressed.

```cpp
template <typename T>
void myMethod(T value) {
    cout << "Generic implementation" << endl;
}

template <>
void myMethod<int>(int value) {
    cout << "Specialized for int" << endl;
}

int main() {
    myMethod(10);       // prints "Specialized for int"
    myMethod(10.5);     // prints "Generic implementation"
    return 0;
}
```

Listing 9. *Specialization in C++. The code is the equivalent of that in Listing* **??**.

```rust
#![feature(specialization)]
trait MyTrait { fn my_method(&self); }

impl<T> MyTrait for T {
    fn my_method(&self) {
        println!("Generic implementation");
    }
}

impl MyTrait for i32 {
    fn my_method(&self) {
        println!("Specialized for i32");
    }
}

fn main() {
    10.my_method();     // prints "Specialized for i32"
    10.5.my_method();   // prints "Generic implementation"
}
```

Listing 10. *Specialization in Rust (unstable). The code is the equivalent of that in Listing* **??**.

3.3 Metaprogramming

Metaprogramming refers to the practice of writing code that generates or manipulates other code, enabling abstraction, automation and domain-specific optimizations. Historically rooted in Lisp's symbolic macro systems [**?**] and later expanded through compile-time techniques such as C++ templates [**?**], metaprogramming has evolved into a fundamental paradigm in modern language design. Its goals are multifaceted: reducing boilerplate, enforcing structural invariants, encoding repetitive patterns and enabling compile-time specialization or reasoning that would otherwise be impossible or inefficient at run-time. Across programming ecosystems, metaprogramming serves both practical and expressive purposes, ranging from code generation and DSL construction to advanced type-level computation. Rust approaches metaprogramming through a powerful and expressive macro system designed to maintain the language's guarantees of safety, determinism and predictable compilation. Rust distinguishes between two primary categories: *declarative macros* and *procedural macros*.

Declarative Macros Declarative macros (defined using `macro_rules!`) operate using pattern matching over token streams, allowing developers to define custom syntactic constructs that expand into valid Rust code. While reminiscent of Lisp macros in spirit, Rust's declarative macros enforce *hygiene* [**?**, **?**, **?**]: macro-generated identifiers cannot inadvertently capture variables or introduce unintended bindings. This prevents entire classes of errors common in textual or non-hygienic macro systems.

Procedural Macros Procedural macros provide direct access to the compiler's abstract syntax tree (AST [**?**, **?**]) through the `proc_macro` API. These macros behave as compiler plugins that transform annotated code items at compile time. Three major forms exist: *function-like procedural macros*, *attribute macros* and *derive macros*. The latter are widely

used to automatically implement traits such as `Clone`, `Debug` or `Serialize`, significantly reducing repetitive boilerplate and enabling library authors to design ergonomic declarative APIs. Attribute macros allow developers to attach custom semantics or transformations to items, facilitating the creation of embedded domain-specific languages (DSLs) and compile-time validation tools; the `#[when(...)]` attribute macro introduced in this thesis is an example of such usage. Function-like procedural macros (e.g. the `spec!` macro) enable arbitrary code generation based on input token streams, allowing for complex compile-time computations and code synthesis. Because procedural macros operate on structured syntax trees instead of raw text, they can enforce well-formedness and preserve strong static guarantees during expansion.

Integration with the Compilation Pipeline Rust's macro system integrates deeply with the compiler's type checking and optimization phases. Unlike languages where macros expand at a separate preprocessing stage (e.g. the C preprocessor [**?**]), Rust expands macros as part of the front-end compilation pipeline, ensuring that hygiene, scoping and type resolution remain consistent. This design enables robust static analyses to occur alongside macro expansion and avoids undesired ambiguities or semantic inconsistencies. The combination of hygienic rewriting, AST-level transformations and compile-time type awareness makes Rust's metaprogramming capabilities both expressive and safe, empowering developers to construct efficient abstractions without compromising reliability.

4
A deep dive into MyTool

4.1 Introduction

In Chapter **??**, we presented a practical example showcasing the usage of MyTool. This chapter dives deeper into its design principles and implementation details, elucidating the underlying mechanisms that enable stable specialization through metaprogramming. The actual implementation is in Rust, but as stated earlier, the concepts can be generalized to other programming languages with similar capabilities. First, we begin by outlining the core design goals that guided the development of the tool.

Design Goals The primary design goals for MyTool were centered around achieving effective specialization in stable code while adhering to the principles of safety, usability and performance. These goals shaped the tool's architecture, macro design and code generation techniques. The key objectives included:

- **Stable Compatibility**: Ensure that the tool operates entirely within the confines of stable Rust, avoiding any reliance on nightly features or unstable APIs.
- **Type Safety**: Maintain Rust's strong type safety guarantees, ensuring that specialized implementations are correctly typed and validated at compile time.
- **Ease of Use**: Provide a straightforward and intuitive interface for developers to define specialized implementations without excessive boilerplate or complexity.
- **Performance Efficiency**: Generate specialized code that is optimized for performance, minimizing overhead and ensuring that the benefits of specialization are realized in practice.

Overview As already seen, MyTool leverages the macro system to enable specialization. The three primary components needed by the approach are:

- **??**: A macro that annotates specialized implementations, specifying the conditions under which they should be used and generates the necessary boilerplate.
- **??**: A second macro that wraps method calls, directing them to the appropriate specialized implementation.
- **??**: A way to analyze the code at compile time and collect the necessary information to facilitate specialization.

4.2 Conditions Definition

The first step to achieve specialization through metaprogramming is to define various specialized implementations that should be applied when certain conditions are met. In MyTool, this is accomplished using the `#[when(predicate)]` attribute macro, which takes as argument the predicate, which is the condition that must be satisfied for the specialized implementation to be selected. Please note that if we wanted a simpler drop-in replacement for the unstable specialization feature, we could edit the tool to avoid passing the predicate altogether and just use normal rust syntax for implementations, but we still would need to mark the implementation with some kind of macro to generate the necessary boilerplate.

Predicates In our implementation, the macro supports these predicates:

- **Type Equality**: Check if a generic type parameter is of a specific type, e.g. `T = i32`.
- **Trait Implementation**: Check if a generic type parameter implements a specific trait or multiple traits, e.g. `T: Display` or `T: Debug + Clone`.
- **Logical And**: Check if all the inner predicates are satisfied, e.g. `all(predicate1, predicate2, ...)`.
- **Logical Or**: Check if at least one of the provided predicates is satisfied, e.g. `any(predicate1, predicate2, ...)`.
- **Logical Not**: Check if the provided predicate is not satisfied, e.g. `not(predicate)`.

These predicates can be combined to form complex conditions, allowing for flexible and powerful specialization rules.

Wildcards In addition to the specific predicates, MyTool supports a wildcard condition that matches any type. For instance, the condition `T = Vec<_>` matches any vector type, regardless of its element type. Another useful wildcard condition is `T = &_`, which matches any reference type.

Default Implementation In addition to specialized implementations, MyTool supports a default implementation that is used when no other conditions are met. These default implementations are picked up by default by the **??** and don't need to be explicitly marked with any macro. Optionally, if we wanted to limit the implementations available for specialization, we could use another macro to explicitly mark the default implementations.

Normalization To ensure that the conditions are consistent and can be easily compared, MyTool normalizes the conditions. This normalization process involves transforming the conditions into a canonical form, which allows for easier comparison and resolution of overlapping conditions. The normal form chosen is a disjunctive normal form (DNF) [**?**], where each condition is represented as a conjunction of predicates. This means that each condition is a logical OR of one or more predicatesand

each predicate is a logical AND of one or more atomic conditions. The normalization process also involves applying De Morgan's laws [**?**] to push negations down to the atomic conditions, ensuring that the conditions are expressed in a consistent manner. This representation allows for efficient checking of whether a condition is satisfied, because we can easily check each disjunctive clause separately. For example, the condition `not(all(T: Display, any(T = i32, T = f64)))` is normalized to `any(not(T: Display), all(not(T = i32), not(T = f64)))`, so we know that that specific specialized implementation should be applied either if `T` does not implement `Display` or if `T` is neither `i32` nor `f64`.

Macro expansion Whenever possible, the conditions are evaluated at compile time and the generic parameters are replaced or constrained accordingly. For example: if we have a type equality condition, the generic parameter is replaced with the specific type; if we have a trait implementation condition, the generic parameter is constrained to implement the specified trait(s). Given that all the conditions are normalized, the macro expansion can be greatly optimized: the `when` macro expands each disjunctive condition into a separate specialized implementation, allowing for efficient resolution of the conditions at compile time. This is the reason why we previously defined this approach *monomorphic specialization*.

Nested Conditions The macro also supports nested conditions, allowing for more complex specialization rules. For example, we can have a condition that checks if a type is a vector of types that implement a specific trait, e.g. `all(T = Vec<U>, U: Display)`. Some particular cases could also be completely flattened during normalization, e.g. `all(T = Vec<U>, any(U = i32, U = f64))` would be flattened to `any(T = Vec<i32>, T = Vec<f64>)` and hence an implementation would be generated for each of the two cases.

Examples Listings **??** and **??** show an example of how to define specialized implementations using the `#[when(predicate)]` macro and the corresponding expansion generated by the macro. The examples generate specializations for a struct `MyType` and a trait `Foo`, which takes a generic type parameter `T` and has only one method `foo(&self, x: T)`, as seen in Listing **??**.

```
1  struct MyType;
3  trait Foo<T> {
4      fn foo(&self, x: T);
5  }
```

Listing 11. *Definition of the simple struct and trait used in the following listings.*

```rust
// Default implementation
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Default Foo for MyType");
    }
}

// Specialized implementation for Display
#[when(T: Display)]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T implements Display");
    }
}

// Specialized implementation for &[_]
#[when(T = &[_])]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is &[_]");
    }
}

// Specialized implementation for not Vec<_>
#[when(not(T = Vec<_>))]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is not Vec<_>");
    }
}

// Specialized implementation for MyI8 or &u8
type MyI8 = i8;

#[when(any(T = MyI8, T = &u8))]
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}
```

Listing 12. *Example of the conditions definition using the tool.*

```rust
// Default implementation
impl<T> Foo<T> for MyType {
    fn foo(&self, x: T) {
        println!("Default Foo for MyType");
    }
}

// Specialized implementation for Display
trait FooDisplay {
    fn foo(&self, x: String);
}

impl<T: Display> FooDisplay<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T implements Display");
    }
}

// Specialized implementation for &[_]
trait FooSlice<T> {
    fn foo(&self, x: &[T]);
}

impl<T> FooSlice<T> for MyType {
    fn foo(&self, x: &[T]) {
        println!("Foo for MyType where T is &[_]");
    }
}

// Specialized implementation for not Vec<_>
trait FooNotVec<T> {
    fn foo(&self, x: T);
}

impl<T> FooNotVec<T> for MyType {
    fn foo(&self, x: T) {
        println!("Foo for MyType where T is not Vec<_>");
    }
}

// Specialized implementation for MyI8 or &u8
type MyI8 = i8;

trait FooMyI8 {
    fn foo(&self, x: MyI8);
}

impl FooMyI8 for MyType {
    fn foo(&self, x: MyI8) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}

trait FooRefU8 {
    fn foo(&self, x: &u8);
}

impl FooRefU8 for MyType {
    fn foo(&self, x: &u8) {
        println!("Foo for MyType where T is MyI8 or &u8");
    }
}
```

Listing 13. *Example of the expansion of the conditions definition defined in Listing* **??**.

4.3 Method Wrapping

To enable the selection of the appropriate specialized implementation at method call sites, we need a second macro, which wraps method calls and directs them to the correct implementation. In MyTool, this is accomplished using the `spec!` macro, which takes as first arguments the method call itself.

Type Annotations If the language supported reflection or if we had some way to inspect the type of the arguments at compile time, this would have been enough. However, Rust does not support such features, so we need to provide additional information to the macro. This additional information includes the types of object on which the method is called and the types of the arguments passed to the method, if any.

- **Type of the Object**: The type of the object on which the method is called, e.g. if we have `x.fn(arg1, arg2)`, we need to provide the type of `x`, e.g. `MyType`.
- **Types of the Arguments**: A list of the types of the arguments passed to the method, if the method takes no arguments, it can be empty or omitted, e.g. if we have `x.fn(arg1, arg2)`, we need to provide the types of `arg1` and `arg2`, e.g. `[Arg1Type, Arg2Type]`.

With this information, the tool can solve almost all of the specialization at compile time, generating a direct call to the appropriate specialized implementation.

Annotations The *type annotations* defined above are the first two of what we call *annotations*, which are additional pieces of information provided to the macro to help it resolve the specialization. In Rust, just like there is no way to inspect the type of a variable at compile time, there is also no way to inspect the traits implemented by a type or to inspect type aliases. To overcome these limitations, MyTool allows the user to provide additional annotations to the method wrapping macro. These additional annotations include:

- **Implemented Traits**: A list of traits implemented by each relevant type, e.g. `i32: Display` or `i32: Debug + Clone`.
- **Type Aliases**: A list of type aliases for each relevant type, e.g. `MyInt = i32`.

These annotations can be provided as additional arguments to the `spec!` macro (without any particular order), allowing the tool to accurately resolve the specialization even in the presence of type aliases or when traits are involved in the conditions. Considering all of this, the full syntax for the `spec!` macro is the following: `spec!{ method_call; var_type; args_types; annotations... }`, where `method_call` is the method call, `var_type` is the type of the object on which the method is called, `args_types` is a list of the types of the arguments passed to the method (if any) and `annotations` are any additional annotations provided by the user. Additionally, for languages that support class inheritance, another annotations could be added to reflect parent classes, e.g. `T1 <: T2`.